

The Event Loop, Blocking Calls, and the SafeChain Thread Pool

2026-07-01

Why a blocking `chain.invoke()` freezes the event loop

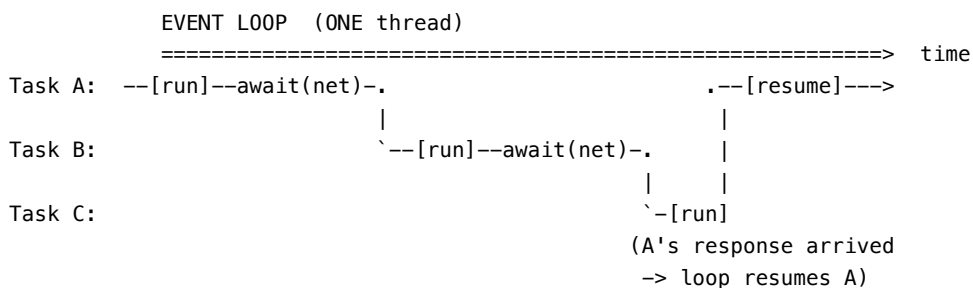
This note explains, with diagrams:

1. how the async **event loop** juggles many tasks on a single thread,
2. why a blocking call like SafeChain's `chain.invoke()` freezes that loop,
3. how exiling it to a **thread pool** fixes it, and
4. a side note: a “thread” is not the same as “a CPU running”.

The core fact to hold onto: **the event loop is a single thread, and it can only run one piece of code at a time**. It switches between tasks *only* at `await` points, where a task voluntarily hands control back.

Diagram 1 — Normal async loop (cooperative, never blocked)

One thread (the event loop) runs many tasks by switching at every `await`. An `await` on a network call means “wake me when bytes arrive — meanwhile, loop, go run someone else.” The network waits happen OFF the loop, so they overlap and everyone progresses.

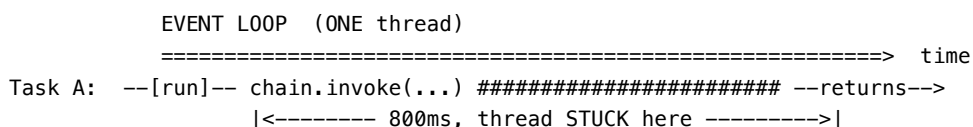


Legend: [run] = loop actively executing this task's code
await(net) = task yields control back to loop; its network wait happens OFF the loop (the OS handles it)

The loop is **never idle while work exists** — the moment a task says `await`, the loop picks up another. The waits overlap.

Diagram 2 — `chain.invoke()` directly on the loop (FROZEN)

`chain.invoke()` has no `await` inside. The loop's single thread enters it and gets **stuck there** for the whole network round-trip. No yield -> no switch -> nothing else runs.



(no await inside -> loop CANNOT switch away)

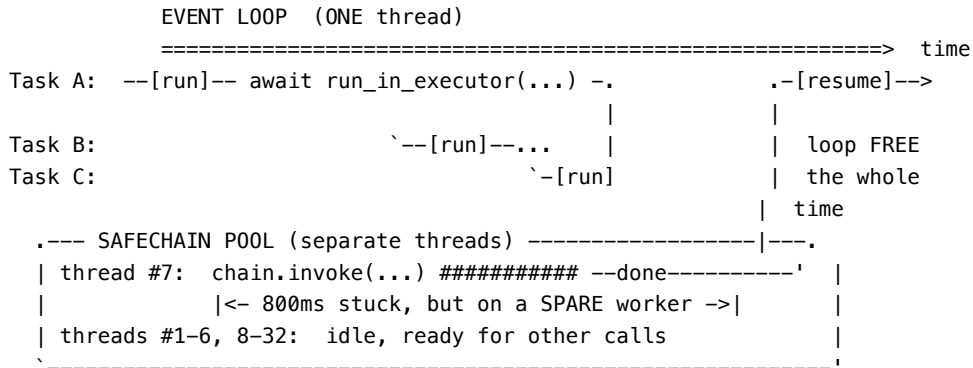
```
Task B:      .... frozen – cannot run
Task C:      .... frozen – cannot run
SSE -> browser: .... silent (user thinks it crashed)
new request: .... can't even be accepted
```

= the one loop thread is captive inside invoke(), doing nothing useful, but refusing to hand control back

Note: the thread is not *computing* during those 800ms — it is just **waiting on the network**, but waiting *synchronously*, so it holds the loop hostage. Same wait as Diagram 1, done in a way that blocks everything.

Diagram 3 — The fix: hand the blocking call to a pool thread

run_in_executor ships chain.invoke to a **separate** worker (one of the 32 SafeChain pool threads). That worker freezes instead — harmless, it is a spare — while the loop **awaits** the handle and stays free.



Code shape (llm/safechain_client.py):

```
model = await amodel(model_id)          # async BUILD (token acquisition)
chain = prompt | model | StrOutputParser()
# ...
await asyncio.wait_for(
    loop.run_in_executor(_SAFECHAIN_EXECUTOR, chain.invoke, ...), # blocking
    timeout=_SAFECHAIN_CALL_TIMEOUT_S,                             # part on
)                                                                    # OUR pool
```

The freeze still happens — it is unavoidable for a blocking call — but it is **exiled to a spare thread** whose only job is to absorb it. The loop never stalls.

Why a killed call can leave a thread occupied

asyncio.wait_for(..., timeout=...) limits the **await** — the *loop's waiting*. On timeout the loop says “I give up waiting” and moves on, freeing the asyncio-side bookkeeping (the task, the firewall semaphore slot, the turn lock). **But the pool thread is still running chain.invoke** — Python cannot interrupt a thread blocked in C/IO, so it stays busy until SafeChain returns on its own. This is bounded: it is one of 32 threads, reused not spawned, and self-draining. (In the dev/OpenAI path the call is natively async, so cancel aborts the request cleanly — no thread lingers.)

Side note: a “thread” is not “a CPU running”

A thread is **not** a CPU, and an “occupied” thread is usually **not** burning a CPU.

A thread is a line of execution with its own little stack of “where am I in the code”. The **operating system** schedules threads onto the physical **CPU cores**. They are not 1:1:

threads (can be hundreds)	CPU cores (e.g. 8 physical)
-----	-----
thread #1 -. .	
thread #2 - OS scheduler	[core 0]
thread #3 -+--- decides who ->	[core 1] (only a handful
thread #7 - runs on a core	[core 2] actually run at
... -' each instant	... any given moment)

So many threads share few cores; the OS rapidly time-slices between them.

The crucial part — a *blocked* thread uses ~0 CPU. When pool thread #7 is stuck in `chain.invoke` waiting for the network, it is **not** running on a core. It is **parked/asleep** — the OS sets it aside and gives the cores to other threads. It consumes:

- **CPU:** basically none (it is waiting, not calculating).
- **Memory:** a little (its stack + the request/response buffers).
- **A pool slot:** yes — and *this* is the real cost. It can’t take a *new* job until it wakes up.

Occupied (waiting on network):	seat taken, engine OFF	← SafeChain case
Busy (crunching numbers):	seat taken, engine ON	

So “a killed distiller occupies a pool thread” does **not** mean a CPU is pinned at 100%. It means **one of the 32 worker slots is checked-out and idle-waiting**, unavailable for other calls, until SafeChain returns. An *occupied seat*, not a *running engine*.

Bonus wrinkle (Python’s GIL). CPython’s Global Interpreter Lock lets only one thread run *Python bytecode* at a time. But blocking I/O calls **release the GIL while they wait**. So while thread #7 is parked on the network, it has let go of the GIL, and the event-loop thread is free to keep running Python. That is *why* the thread-pool trick works at all: the blocking thread sleeps (GIL released, no core, no Python), and the loop keeps serving everyone.

Summary: thread != CPU. A thread is a schedulable worker; whether it uses a CPU depends on whether it is *computing* (yes) or *waiting* (no). The SafeChain “occupied” thread is the waiting kind — costing a slot and a little memory, not a core.